

Mini-kviz: *Koje greške mogu da se dese?*

Pre nego što počnemo sa ovim mini-kvizom, želim da vam dam nekoliko dodatnih, a vrednih, objašnjenja:

- **Sistemske greške:** (ne postoji fajl, prava pristupa, puna memorija) imaju broj greške [Errno X]. **Logičke/Format greške:** (loš JSON, deljenje nulom, pogrešan tip) imaju samo tekstualni opis.
- U starim verzijama JSON standarda, bilo je obavezno da JSON fajl sadrži objekat - rečnik {} ili listu []. Ali moderni standard **dozvoljava da bilo koji validni tip podatka bude samostalni JSON fajl**. To znači da je sasvim validno da JSON fajl sadrži int, float, string, Boolean, null.
- Kako se obezbediti da ako JSON fajl sadrži nešto što nije rečnik ili lista? – Postavljamo jedan “detektor” ☺ : To je f-ja **isinstance()** . Primer:

```
sadrzaj = json.loads(file.read_text())
# Ako je u fajlu rečnik, string, ili broj... a treba nam lista
if not isinstance(sadrzaj, list): # Ako promenljiva sadrzaj nije lista
    sadrzaj = []                 # Isprazni i postavi kao listu
ILI
if not isinstance(sadrzaj, (list, dict)): # stavimo list i dict u torku i tretiramo ih oboje, redosled nebitan
    print("Ovo nije ni lista ni rečnik, nešto nije u redu sa fajlom!")
ILI
# Ako proveravamo i svejedno nam je koji je tip, samo da je "jedan od onih koji prolaze"
if isinstance(sadrzaj, (str, int, float, bool, list, dict)):
    # Može u JSON!
```

IDEMO NA KVIZ:

Cilj domaćeg zadatka

Cilj ovog zadatka je da:

- otkriješ **koje greške mogu nastati** pri radu sa JSON fajlovima
- naučiš kako da **prepoznaš i razumeš greške** u Python-u.

Odgovore upisuj u notes svog mobilnog, pod rednim brojevima zadataka, pa ćemo proveriti na času.

Početni program

Analiziraj i pokreni program (ima ga na GitHub-u kao: *json_domaci_otkrij_gresku.py*):

```
import pathlib
import json

file = pathlib.Path("otkrivanja.json")

try:
    contents = file.read_text(encoding="utf-8")
    sadrzaj = json.loads(contents)
    print("Sadržaj fajla:", sadrzaj)
except Exception as e:
    print("Greška:", e)
```

Objašnjenje:

- **Exception:** *Exception* je **uopšteno** za sve uobičajene greške u Pythonu (deljenje nulom, pogrešan tip podatka, fajl ne postoji, itd.). Kada napišeš **Exception** kažeš Pythonu: *"Uhvati bilo koju standardnu grešku koja se desi"*.
- **as e** : Ovo je kreiranje nadimka (promenljive). Python uzme detalje o grešci koja se upravo desila i "spakuje" ih u promenljivu koju smo nazvali "e" i onda tu promenljivu možemo da koristimo.
- **Izbegavamo da stavljamo u svoj kôd samo except ili except Exception** . (Ovo je samo za igru - kviz).

ZADATAK 1: Kada fajl ne postoji

1. Obriši fajl *otkrivanja.json* (ako postoji)
2. Pokreni program

Odgovori:

- Koja greška se pojavila?

- Kada se ova greška dešava?

ZADATAK 2: Prazan fajl

1. Napravi prazan fajl *otkrivanja.json* (otvori ga i ne upisuj ništa)
2. Pokreni program

Odgovori:

- Koja greška se pojavila?
- Zašto dolazi do ove greške?

ZADATAK 3: Neispravan sadržaj

1. U fajl otkrivanja.json upiši nešto pogrešno, na primer:

abc ili: {ime: Lena}

2. Pokreni program

Odgovori:

- Koja greška se pojavila?
- Šta nije u redu sa sadržajem fajla?

ZADATAK 4: Ispravan JSON

1. U fajl upiši ispravan JSON, na primer:

5 ili: "Zdravo"

2. Pokreni program

Odgovori:

- Da li se pojavila greška?
- Šta program sada radi?

ZADATAK 5: Zaključivanje

Na osnovu prethodnih eksperimenata:

Dopuni kod tako da hvata samo očekivane greške (pomoć: obe vrste grešaka navedenih u *except* imali smo u JSON lekciji, u kodu koji sadrži odvojene funkcije):

```
except (_____, _____):  
    print("Podaci nisu dostupni ili nisu ispravni.")
```

PITANJE

Zašto nije dobro koristiti samo:

except :

Odgovor: *(dovoljno je da se navede 1–2 razloga)*

Zaključak

Koje dve najčešće greške mogu nastati pri radu sa JSON fajlovima? (Može i svojim rečima, mada ih imamo u Zadatku 5):

- 1.
- 2.